T01

T01 — Service Boundaries and Architecture Styles

Complete reference learning guide | Beginner → Senior Solution Architect interview depth
Syllabus: `solution_architect_interview_topic_map_v3.md` (authoritative, revision 2026-10-24)
Reference verification date: 2026-10-24
Mode: Self-study reference. **Do not begin interactive teaching or mark the learner as passed until explicitly requested.**

DOCUMENT NAVIGATION

Contents

Click any heading to jump to that section. Page numbers refer to this PDF edition.

01 / Orientation & prerequisites

0. How to use this guide and what it covers 3
P0. Prerequisite bridge — enough F02 and foundations for T01 4
Running case and stated assumptions 6

02 / Fifteen progressive lessons

L01. The difference between a code boundary and a service boundary 7
L02. Layered architecture and multi-tier deployment 7
L03. Service-oriented architecture (SOA) 8
L04. Modular monolith 9
L05. Microservices 10
L06. Event-driven designs 11
L07. Compare all styles without confusing their dimensions 12
L08. Find service candidates from business capabilities 13
L09. Place the authoritative data and write boundaries 14
L10. Test boundaries using ownership and rate of change 14
L11. Turn a proposed boundary into an explicit decision 15
L12. Tenants and isolation across service boundaries 17
L13. Model company hierarchy and policy inheritance 18
L14. Shared platform capabilities versus product-specific behavior 19
L15. Build, buy, or reuse: the boundary is also an ownership choice 20

03 / Demonstration, practice & assessment

16. Runnable Python demonstration — modules, tenants, hierarchy, and ownership 21
17. Cumulative architecture exercise — the T01 practice task 25
18. Senior decision challenge — answer before reading §20 26
19. Practice questions — do these without notes 27
20. Separate answer key — exercise solutions and model decisions 29
21. Assessment rubric and completion criteria 35

04 / Sources & completeness audit

22. Official primary references and verification record 37
23. Final completeness and verification audit 38

0. How to use this guide and what it covers

> **Source requirement (topic map):** T01 has three **Core** items: **C036**, **C037**, and **C038**. The topic map labels **F02** as T01's prerequisite. **F02 actually owns C019–C022**, not C036–C038. This guide teaches all T01 items in detail and reviews only the F02 knowledge required to understand T01. It does not silently add other cards to T01's required scope.

What success looks like

After completing the lessons, you can (1) compare layered / multi-tier, service-oriented architecture (SOA), modular monolith, microservices, and event-driven designs; (2) draw defensible service boundaries using business capabilities, ownership, rate of change, and data authority; and (3) defend tenant isolation, company hierarchy, platform sharing, and product-specific differences. You can **explain, apply, and defend a trade-off without notes**. Merely knowing the names is not enough.

Follow this order: Understand → Explain → Demonstrate → Practice → Evaluate → Memorize. In each short lesson, read the example, do the exercise without looking at the answer key, examine the failure case, and practice the spoken answer. Take the senior decision test only after the lessons. **All answers and model decisions are in §20, not beside the exercises.**

Evidence labels used throughout

- **Source requirement:** A binding rule or scope statement from the supplied v3 map. This is the syllabus, not an external technical standard.
- **Verified external fact:** A factual or documented architecture concept supported by a **primary publisher/ standards organization**, cited as `[R#]` and listed in §22. "Verified" means the linked primary reference was checked on **2026-10-24**; it does not mean a pattern is always the best choice.
- **Hypothetical practice assumption:** An invented detail for learning. It is **not** a claim about American Express or another company.
- **Recommendation:** Design judgment. Apply only after checking business goals, risks, team ability, and evidence.

Scope boundary: T01 introduces the *reason* for cross-service calls and event use. Deep retries, ordering, exactly-once claims, and distributed transactions are **T02**; advanced event contracts and broker recovery are **T04**; detailed authentication and encryption are **T06**; enterprise migration execution is **E01**. These are named but not taught as new mandatory T01 checklist items.

Coverage matrix — authoritative checklist crosswalk

ID / level	Exact required topic	Dedicated lessons	Direct practice evidence	Assessment evidence
C036 · Core	Compare layered/multi-tier systems, SOA, modular monoliths, microservices, and event-driven designs	**L02–L07** (L01 vocabulary)	E02–E07; style comparison worksheet	Q01–Q05, Q13; cumulative task §17 part A; senior test §18
C037 · Core	Derive service boundaries from capabilities, ownership, change patterns, and data boundaries	**L08–L11** (L01 foundations)	E08–E11; two decompositions, owner/data map	Q06–Q09, Q14; cumulative task §17 parts B–C; senior test §18

ID / level	Exact required topic	Dedicated lessons	Direct practice evidence	Assessment evidence
C038 · Core	Account for tenant isolation, organizational hierarchy, shared platform capabilities, and product-specific behavior	**L12–L15**	E12–E15; negative isolation tests and inheritance table	Q10–Q12, Q15; cumulative task §17 parts D–E; senior test §18
F02 — prerequisite only	Architecture views, option comparison, traceability, reviews (**C019–C022**)	**P0** recap; L11 applies a small ADR	P0 drawing and traceability check	Refer to the **F02 guide** for full F02 evaluation; not counted as T01 coverage

Audit principle: "Covered in this document" and "mastered by the learner" are different statuses. The second requires assessed work.

P0. Prerequisite bridge — enough F02 and foundations for T01

Purpose. Before drawing boundaries, understand what software, a request, stored data, and an architecture diagram are. A prerequisite may be familiar from B01–B03, F01, D01, and F02; the definitions below are sufficient to read this guide, but **do not replace completing those prerequisite cards**.

Vocabulary, from zero:

Term	Meaning	Tiny example
Program	Instructions written for a computer	Python code checking a spending rule
Process	A running instance of a program	One running backend application
Module	A group of related code	`policy` code together
Function	A named piece of code receiving inputs and returning output	`is_allowed(amount, limit)`
Server / backend	Software handling requests from other software	Purchase request handler
Client	Software making a request	Employee web application
Application programming interface (**API**)	Defined contract for software communication	"Submit purchase" operation; accepted fields and results
Database / table / record	Durable organized data / named collection / single entry	`policy` table / limit for Branch A
Database transaction	Group of database changes that commit together or roll back together	Write an approved rule and its audit entry
Business capability	What an organization **does** to create value	Manage spending rules
Requirement	What the system must do	Only a permitted administrator may change a limit
Quality attribute	How well the system must behave	Correctness, availability, changeability, data isolation
Architecture	Important structural decisions: responsibilities, connections, ownership, and constraints	Policy module owns policy rules

Term	Meaning	Tiny example
Component	Part of a system with a defined job	Purchase processor
View / diagram	A picture answering one architecture question	Context view: what connects to the platform?
Boundary	Line separating responsibilities, data authority, trust, or deployment	Policies vs. purchases
Stakeholder / owner	Person affected by / accountable for a decision or capability	Finance / policy product owner
Trade-off	A choice that improves one quality while costing another	Separate deployment improves release autonomy, adds network risk
Deployment / release	Putting a runnable software version into an environment / making a new version available	Deploy Policy with Purchase, or independently
Latency	Time between a request and its observed result	Delay before an eligibility answer appears
Scale / scaling	Ability / action to handle more demand by adding or adjusting resources	More purchase requests at peak
Architecture decision record (**ADR**)	Short document recording a decision, options, consequences, and review trigger	“Start with a modular monolith”

Worked example — three small views, not an exhaustive F02 class:



Step-by-step: The context picture identifies actors and the entire solution. The component picture separates jobs. The sequence picture shows a decision over time. These pictures answer different questions, so no single picture replaces all three.

P0 check (not graded as T01): Draw the three views above from memory, and explain (a) where a purchase decision occurs, (b) which component owns the rule, and (c) which business requirement each arrow supports. If these are unfamiliar, revise F02 before attempting the senior T01 test.

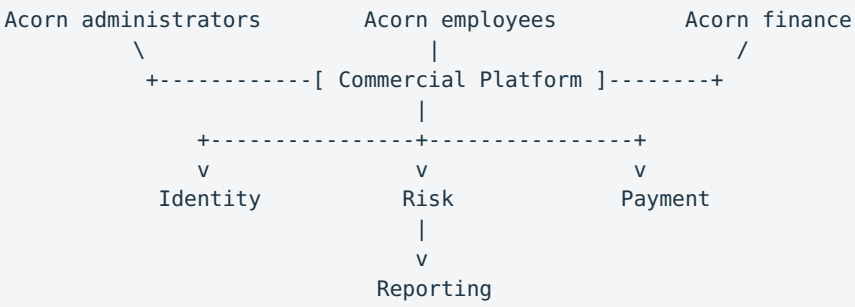
Common mistake: Treat a drawing as proof of a good design. Diagrams still need responsible owners, data authority, tests, and failure decisions.

> **Interview Phrasing:** “I use a context view to show the solution boundary, a component view for responsibilities, and a sequence view for the critical business flow. I check each view against a requirement.”
> *Plain English:* Use the right diagram to answer the right question.
> *Follow-up:* “Which view would expose a missing data owner?” → The component/data ownership view, supplemented by a responsibility table.

Running case and stated assumptions

> > **Hypothetical practice assumption — applies to every lesson unless replaced in an exercise:** We are designing a **fictional** commercial-spending platform. A **tenant** is a customer business whose users and information must be separated from other customer businesses. An **administrator** configures spending policies (rules such as “up to 500 units”). An **employee** requests a purchase. A **finance user** checks transaction records against the organization's accounting records; this is **reconciliation**. The platform also communicates with identity (who a user is), risk (whether an action looks unsafe), payment (external financial execution), and reporting (summaries). A “unit” is invented training currency, **not an actual exchange rate or payment limit**.

> **Further practice assumptions:** Two fictional business customers exist, **Acorn** (Tenant A) and **Birch** (Tenant B). Acorn has Headquarters, Sales, and Operations departments. For toy diagrams, Acorn's Headquarters gives a base spending limit of **500 units**; Sales may reduce it to **200 units**. Both products “Employee Spend” and “Supplier Pay” use identity, but they have different business approval rules. The demo has **two engineering teams**. These numbers and arrangements are invented, not factual performance or organizational data. No company-specific service boundaries, private payment implementation, or official interview round is claimed.



L01. The difference between a code boundary and a service boundary

Mapped items: foundation for C036–C038. **Purpose/prerequisite:** Learn this before comparing architecture names; requires P0 terms.

Terms and how it works: A **code boundary** separates modules and their permitted interactions inside code. A **runtime boundary** separates running processes. A **deployment boundary** separates things that can be released independently. A **data boundary** separates who may write authoritative facts. A **trust boundary** separates areas with different access rights. A **service boundary** usually combines a coherent business responsibility, an externally usable contract, ownership, and sometimes independent runtime/deployment. A service is *not* automatically a microservice. **Cohesion** means related behavior stays together. **Coupling** means one part depends on another: more coupling usually means more coordinated changes.

Example: A `Policy` module and a `Purchase` module can run in **one** Python process with clear code and data ownership. If deployed in separate processes, the policy check needs a communication boundary and a failure policy. Neither design is automatically safer.

```
Code separation inside ONE running application:
[ Application process: Policy module | Purchase module ]

Independent deployed services:
[ Purchase process ] -- network call --> [ Policy process ]
```

Step-by-step: In the first case the modules call local functions and usually release together. In the second, they must agree on a contract, communicate, and handle network unavailability; they may release independently if genuinely decoupled.

> **E01 — guided exercise:** Draw the same Policy/Purchase split with (a) one process and (b) two processes. Label code, runtime, deployment, data, and trust boundaries. **Expected:** Five kinds of boundary are distinguishable. **Hint:** Independent code folders do not prove independent deployment.

Failure/trade-off: "One module = one microservice" creates too many processes. Alternatively, "one deployment = no boundaries" makes tangled code. Measure coordination pain before paying for distribution.

> **Interview Phrasing:** "I separate logical ownership from physical deployment. I can keep firm module and data boundaries within one deployable unit, then split deployment only if the business and team need it."
> *Plain English:* Good separation is possible without many services.
> *Follow-up:* "Can microservices share one database?" → They might share a database server, but shared writable tables couple their models and undermine data autonomy [R03].

L02. Layered architecture and multi-tier deployment

Mapped: C036. **Purpose/prerequisite:** Learn a common simple structure before services. Requires L01.

Terms: A **layer** is a logical group by type of job. **Presentation layer** handles user input/output. **Business layer** applies rules. **Data access layer** reads/writes storage. A **tier** is a separately deployed/runtime part, often on a different server or process. A three-layer design can run in one deployable application; "three layers" does **not** necessarily mean "three servers." An **N-tier architecture** organizes logical layers and physical tiers [R01].

How it works and worked example:

```
Browser  --> [ Presentation / HTTP API ]
                        |
                        v
             [ Business: check spend limit ]
                        |
                        v
             [ Data access: load policy ]
                        |
                        v
                  [ Database ]
```

1. Employee submits an amount. 2. Presentation validates the request shape. 3. Business logic compares it with the rule. 4. Data access loads the right customer's rule. 5. Presentation returns allowed/denied. The database can be a separate physical tier even if all other layers run together.

> **E02 — guided exercise:** Assign these actions to layers: parse purchase request, compare with limit, retrieve policy row, render response. Redraw as (a) one application process plus database, (b) web tier plus business tier plus data tier. **Expected:** Layer responsibility and deployment choice are not confused. **Hint:** A tier is about where code runs, not what job it performs.

Failure/trade-off: A "business" layer containing rules for every product may become one large change bottleneck. Layered architecture is easy to reason about, but grouping only by technical function can cause one business change to touch many layers [R01].

> **Interview Phrasing:** "A layered design separates presentation, business rules, and data access. I would use it where one coordinated release is acceptable, but make sure important business capabilities do not become tangled across layers."
> *Plain English:* Simple organization is useful; it does not solve ownership by itself.
> *Follow-up:* "Does N-tier imply microservices?" → No. Tiers are deployment separation; microservices emphasize independently owned business capabilities.

L03. Service-oriented architecture (SOA)

Mapped: C036. **Purpose/prerequisite:** Understand coarse-grained services used by different applications before the more specific microservices style. Requires L02 and the API definition.

Terms: Service-oriented architecture (SOA) organizes capabilities as services with clear **service contracts** (what operations and information they expose). A **consumer** calls a service. A **provider** implements it. **Interoperability** means systems can work together despite different implementations. A **coarse-grained** service covers a larger business responsibility. **Governance** means rules and owners for service changes. OASIS defines an abstract SOA reference model that is not tied to one protocol or technology [R02]. **Important:** SOA does not *require* one enterprise service bus or one specific message format.

Worked example:

```
[Employee Spend] -----------+
                            +--> [Shared Identity Service]
[Supplier Pay] -------------+
                            +--> [Organization Directory Service]
```

1. Both products need user identity. 2. One shared identity service offers a documented interface. 3. The two products can have different workflows and still reuse identity. 4. The shared provider must govern changes for both consumers.

> **E03 — guided exercise:** Identify one coarse-grained shared service, two consumers, its service contract, and the accountable provider owner. **Expected:** Consumers do not read the service's tables directly. **Hint:** Start with “resolve user identity” rather than “UserController class.”

Failure/trade-off: A shared service can become a bottleneck or create a large blast radius if every team depends on one team. The opposite—copying identity code everywhere—creates security and maintenance drift. Place the boundary where reuse justifies the shared contract and support burden.

> **Interview Phrasing:** “SOA promotes reuse through service contracts across applications. I would centralize genuinely shared capabilities while controlling dependency and change impact; I would not assume that SOA means an enterprise bus.”
> *Plain English:* Share capabilities through stable interfaces, but shared ownership has costs.
> *Follow-up:* “How does SOA differ from microservices?” → SOA is a broad service-orientation approach; microservices usually emphasize smaller bounded business contexts and autonomous deployments. Neither label determines quality.

L04. Modular monolith

Mapped: C036. **Purpose/prerequisite:** Learn how to isolate business responsibilities **inside** one application. Requires L01–L03.

Terms: A **monolith** is a system packaged and deployed as one main application unit. A **modular monolith** deliberately separates cohesive business modules within that unit and limits how they call each other. An **internal interface** is a permitted way one module uses another. A **shared database engine** is possible, but tables still need explicit write owners. A **module dependency rule** says which modules may depend on which.

Worked example:

```
ONE deployable application / one release pipeline
+-------------------------------------------------------+
| [Purchase module] -- calls --> [Policy module]        |
|         |                          |                  |
|   [Purchase tables]          [Policy tables]          |
+-------------------------------------------------------+
```

1. Policy module changes policy rules and owns its tables. 2. Purchase module asks Policy for a decision. 3. Purchase module must **not** change policy tables directly. 4. The application starts and deploys as one unit.
Recommendation: Begin with this option when team coordination is manageable and independent deployment has not earned its cost. This is a conditional recommendation, not a universal rule. Modular design can help both monoliths and microservices [R12].

> **E04 — guided exercise:** Define four rules that stop Purchase from reaching directly into Policy internals. Identify one advantage and one disadvantage of a single deployment. **Expected:** Clear interface, owned writes, testable modules, no circular dependency. **Hint:** A folder name alone does not enforce isolation.

Failure/trade-off: Teams may bypass interfaces because local calls are easy, turning a modular monolith into a tangled monolith. A single release can slow independently changing teams, but local function calls, simple testing, and fewer moving parts often lower operational cost.

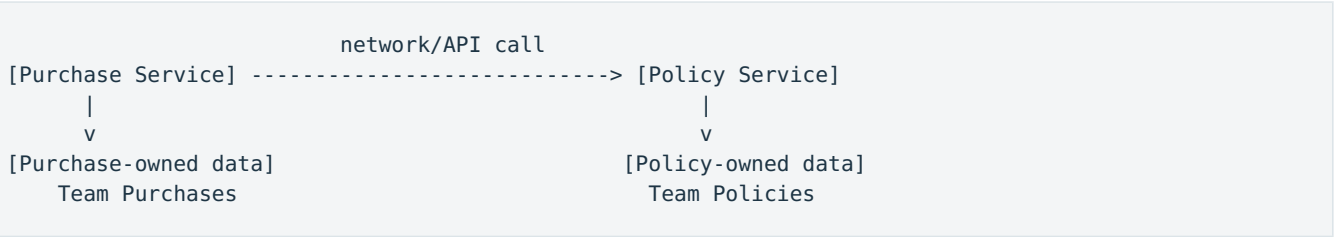
> **Interview Phrasing:** "I would consider a modular monolith if business capabilities can be kept cohesive without separate deployment. It reduces distributed-system overhead, but I would enforce module interfaces and data ownership so we can extract a service later if justified."
> *Plain English:* Keep internals clean now; separate deployment only when needed.
> *Follow-up:* "When would you extract Policy?" → When changes, ownership, or scaling regularly conflict with Purchase and the interface is stable enough to operate separately.

L05. Microservices

Mapped: C036. **Purpose/prerequisite:** Understand the benefit and cost of independently deployed business services. Requires L04.

Terms: A **microservice** is an independently operated service organized around a business capability within a **bounded context** (a place where a business term has one consistent meaning). A **contract** defines how other services use it. **Autonomy** means a team can change a service without forcing coordinated changes elsewhere. **Observability** means being able to understand behavior from signals such as logs and measurements. A **distributed system** has components in different processes communicating through a network; the network can fail. Official Microsoft guidance emphasizes business-domain services, private data, and independent deployment, while documenting communication and operational complexity [R03].

Worked example:



1. Purchase receives the employee request. 2. It asks Policy to evaluate rules through a contract. 3. Each service writes its own records. 4. Teams can release on different schedules if contracts remain compatible. 5. A Policy outage now affects purchase decisions, unless a safe failure policy is designed. **Critical caution:** "Separate deployment" does not automatically mean "independent change."

> **E05 — guided exercise:** Give three conditions that would justify separate Policy and Purchase deployments, and three new responsibilities created by the split. **Expected:** Reasons tied to measured change/ownership rather than fashion. **Hint:** Think deployment frequency, independent scaling, contract management, outages, on-call, and data synchronization.

Failure/trade-off: A request that crosses several services increases calls, uncertain outcomes, and investigation work. A shared database with write access by all services breaks the intended autonomy. Choose microservices only when independence is worth those costs [R03, R04].

> **Interview Phrasing:** “Microservices are useful when bounded capabilities have independent owners and change or scale differently. The gain is team and deployment autonomy. The price is network failure, contract coordination, data consistency work, and operations.”
> *Plain English:* Microservices buy independence by accepting a harder system to run.
> *Follow-up:* “Would you make a separate service for each database table?” → No. Boundaries follow business behavior and ownership, not tables.

L06. Event-driven designs

Mapped: C036. **Purpose/prerequisite:** Understand an interaction style often **combined** with the earlier structures. Requires L03–L05.

Terms: An **event** is a message that describes something that already happened, such as `PurchaseRecorded`. An **event producer** emits it. An **event consumer** reacts. A **broker** is intermediary infrastructure that carries events. **Asynchronous** means the producer does not normally wait for the consumer to finish. **Eventual consistency** means two copies or views of information can temporarily disagree and align later. A **command** asks for work to be done; an **event** reports a fact. Microsoft's event-driven guidance describes producers, channels, consumers, and trade-offs including ordering, recovery, and consistency [R05].

Worked example:



1. Purchase records an approved action in its authoritative store. 2. It makes a `PurchaseRecorded` fact available to subscribers using an appropriate reliable publishing design (**the exact mechanism is T02/T04**). 3. Reporting and notifications process it later. 4. Reporting may temporarily lag. **Do not use a past-tense event to ask another service to perform an uncertain financial action.**

> **E06 — guided exercise:** Classify each statement: `EvaluatePurchase`, `PurchaseRecorded`, `SendNotice`, `LimitChanged`. Choose which belongs in an immediate user-facing decision and which may be sent later. **Expected:** Commands and event facts are not confused; freshness is discussed. **Hint:** Can the fact truthfully be said to have happened already?

Failure/trade-off: An event can arrive late, twice, or not become visible when a publisher crashes unless suitable durability controls exist. Events reduce direct coupling but add monitoring, recovery, privacy, and data freshness questions. Never assume “event-driven” means “microservices” or “guaranteed exactly once.”

> **Interview Phrasing:** “I would use events to notify loosely coupled consumers after an authoritative business action. For an immediate approval decision, I would keep a clear request/response contract; delayed reporting must have an explicit freshness requirement.”
> *Plain English:* Events fit after-the-fact notifications; a customer may still need a direct answer now.
> *Follow-up:* “Could an event-driven application be a monolith?” → Yes. Internal or external events describe communication style, not the number of deployments.

L07. Compare all styles without confusing their dimensions

Mapped: C036 synthesis. **Purpose/prerequisite:** Make a reasoned style choice only after L02–L06.

Key point: These styles are **not mutually exclusive**. Layering describes code organization; tiers describe deployment topology; SOA and microservices describe service-oriented system organization; modular monolith describes packaging and internal discipline; event-driven describes how parts communicate. A layered, modular monolith can publish events. Microservices can use layers internally and consume events [R01, R05].

Choice	Main organizing rule	Typical advantage	Main cost / danger	Strongest reason to consider
Layered / N-tier	Separate presentation, business, data jobs	Simple mental model	Capability changes cut across layers	Stable, straightforward system
SOA	Reusable business services with contracts	Cross-product reuse	Shared governance and dependency	Several applications need a stable capability
Modular monolith	Business modules, one deployment	Local calls and lower operations cost	Joint release and discipline needed	Limited team capacity or limited independent-change need
Microservices	Business contexts with independent services	Release and scale autonomy	Distributed failure and data complexity	Clear independent ownership and measurable autonomy need
Event-driven	Producers announce facts to consumers	Loose coupling for downstream reactions	Delay, duplicates, recovery/visibility	Several consumers react after a business fact

Worked comparison: In the hypothetical platform, initial Policy and Purchase modules might share one deployment, while reporting uses asynchronous events. Later, a policy team that changes rules daily while purchase releases rarely may justify a separate Policy service. This change depends on measured friction and safety tests, not on a fixed industry "best style."

> **E07 — guided exercise:** Score a **modular monolith with events** against **independent Policy and Purchase services with events** on six dimensions: team independence, data ownership, change frequency, response latency, operational overhead, and migration difficulty. Use Low/Medium/High or 1–5 as explicitly **subjective practice ratings**. State what evidence would replace your estimates. **Expected:** A conditional decision and a reversal trigger. **Hint:** Different teams do not necessarily require separate deployment.

Failure/trade-off: A diagram that mixes "layer," "service," and "deployment" into one box obscures where independent releases and network calls actually occur. Draw them separately.

> **Interview Phrasing:** "I treat architecture styles as tools rather than labels. I compare release autonomy, data ownership, call-path latency, operations, and change risk against the business goal. A hybrid design is often valid."
> *Plain English:* Different parts can use different patterns for clear reasons.
> *Follow-up:* "Which style wins?" → None by default; ask what must change independently and what failures the business can tolerate.

L08. Find service candidates from business capabilities

Mapped: C037 — **capabilities**. **Purpose/prerequisite:** Translate the business into stable responsibilities before choosing processes or APIs. Requires P0 and L01–L07.

Terms: A **business domain** is the area of business problems the system handles. A **subdomain** is a smaller business area. A **capability map** groups things the business must be able to do, regardless of technology. **Domain-driven design (DDD)** is an approach that models software using business concepts and their meanings. A **bounded context** is a boundary where terms and rules have one consistent meaning. A **candidate service** is a possible boundary, not yet an approved independent deployment. Domain analysis is not a mechanical process; it needs requirements, team discussion, and iteration [R04].

How it works: Ask "What can the business do?" before "Which table or API exists?" Split responsibilities when they have different rules, owners, and definitions. Prefer high **cohesion** within a capability and manageable **coupling** across capabilities.

Worked example — initial capability map:

```
Commercial Spending
+-- Manage organizations (companies, departments, employees)
+-- Manage spending policy (limits, who may edit rules)
+-- Process purchase requests (receive, record, return outcome)
+-- Support financial execution (connect to external payment capability)
+-- Provide reports (views for finance)
```

1. Policy rules change the meaning of "eligible" or "within limit." 2. Purchase processing owns the request and its current processing state. 3. Reporting presents views but does **not** become the owner of source purchase decisions. 4. Organizational directory owns relationships among business units. This is a **hypothetical candidate map**; requirements could cause a different grouping.

> **E08 — guided exercise:** For this journey—admin sets a limit, employee requests a purchase, finance reviews outcomes—(a) identify five verbs and their business objects, (b) group them into at most five capabilities, and (c) explain why `Policy` and `Purchase` could be different bounded contexts. **Expected:** Capabilities, not controller names. **Hint:** "Who can change the rule?" differs from "What happened with the request?"

Failure/trade-off: Splitting every small noun into a service creates chatty calls. Grouping every financial concept into "Core Service" hides different rules. **Recommendation:** Start with a manageable capability map and test whether a change can be implemented largely inside one candidate boundary.

> **Interview Phrasing:** "I first map business capabilities and the language each team uses. I propose a boundary where rules and responsibilities are cohesive, then test it against data, change patterns, and operational needs."
> *Plain English:* Boundaries follow business meaning, not arbitrary technical parts.
> *Follow-up:* "What is the boundary between policy and purchase?" → One defines eligibility rules; the other owns processing of requests and their state.

L09. Place the authoritative data and write boundaries

Mapped: C037 — **data boundaries**. **Purpose/prerequisite:** Prevent two components from giving competing answers about the same fact. Requires B03 definitions in P0 and L08.

Terms: An **authoritative source** (system of record) is the designated source for a business fact. A **write owner** is the only component authorized to make authoritative changes. A **read model** is a view optimized for reading, possibly delayed. A **schema** is the shape of stored data. **Encapsulation** means hiding internal details behind a defined contract. A **cross-boundary invariant** is a business rule that must remain correct across two owners; an **invariant** is a condition that must always hold where it is enforced. An **audit trail** is a history of important changes and who made them.

Worked ownership table (fictional):

Fact	Authoritative owner	Who may read	Who may change	Boundary interaction
Acorn Sales spend limit	Policy	Purchase, Admin UI	Policy only	Evaluate policy or read through Policy contract
Purchase request state	Purchase	Finance, Reporting	Purchase only	Query Purchase contract or consume published fact
Employee → department membership	Organization Directory	Policy, Purchase	Directory only	Lookup with a trusted identity context
Report summary	Reporting	Finance	Reporting updates derived view	May be stale; not a source transaction

Step-by-step: If Finance sees an old result, it checks the Purchase source record rather than writing a new source status into Reporting. If Purchase needs a limit, it asks Policy rather than editing Policy's table. The physical database server may be shared initially; **ownership must still be exclusive at the logical level** [R03].

> **E09 — guided exercise:** Draw a write arrow and a read arrow for each row. Identify one violation: a reporting process that updates `purchase.status`. Explain the defect and how to test the owner-only rule. **Expected:** One authoritative writer per fact, justified exceptions only with an explicit governance decision. **Hint:** A read cache does not become a source of truth.

Failure/trade-off: A policy check and purchase record may need consistent decisions at the moment of purchase. Keeping them inside one transaction boundary can be simpler, but a separate Policy service may cause a snapshot/version decision and an uncertain result. The full distributed-consistency design belongs to T02. **At T01 level, record which fact and version each owner needs and the failure question.**

> **Interview Phrasing:** "Each critical business fact needs an authoritative owner and controlled update path. I do not let consumers change another capability's tables directly, because that creates hidden coupling and audit gaps."
> *Plain English:* Agree who can change each important fact.
> *Follow-up:* "Can two services use the same database instance?" → Yes, but each should own its tables and access contract; physical sharing is not joint business ownership.

L10. Test boundaries using ownership and rate of change

Mapped: C037 — **ownership and change patterns**. **Purpose/prerequisite:** A sensible boundary must work for people delivering and supporting it, not only for data modeling. Requires L08–L09.

Terms: Accountability means one named role owns an outcome. **Change cadence** is how often a capability changes. **Deployment autonomy** is the ability to release without forcing other teams to release. **Co-change** means components often change together for one feature; this suggests a poor split or a genuine shared rule. **Blast radius** is how much a defect or outage can affect. **Operational ownership** covers monitoring, support, escalation, and recovery.

Worked example:

Fictional evidence to collect	Policy	Purchase	Boundary implication
Who approves changes?	Controls product owner	Purchasing product owner	Different decision paths support separation
How often do changes occur?	Unknown until measured	Unknown until measured	**Do not invent actual release frequency**
How often must both change together?	Collect change-history examples	Collect change-history examples	Frequent coordinated changes weaken autonomy
Who responds to a failure?	Name a controls on-call owner	Name a purchase on-call owner	Separation requires two clear support contracts
Does failure block the user?	Decision might become unavailable	Purchase might be unable to proceed	Set explicit safe failure behavior

How it works: A strong boundary tends to allow most changes inside one capability. Ask product and engineering owners to inspect a sample of recent changes. If almost all require cross-team coordination, separate deployment may add cost without autonomy. If policy changes can be safely released without purchase changes, separation has a clearer payoff. Microsoft advises not to let an organizational chart alone dictate service design [R04].

> **E10 — guided exercise:** Assume, **only for this exercise**, Policy changes on most weekdays while Purchase changes monthly. State two reasons this may support independent deployments, two facts that could overturn the idea, and a responsible owner for validation. **Expected:** A conditional—not absolute—decision. **Hint:** What if the two teams must always change one common shared database table?

Failure/trade-off: Conflating team boundaries with business boundaries creates political or accidental services. Conversely, one overloaded team owning unrelated capabilities can make even a modular monolith hard to change. Assign owners intentionally and revisit boundaries when evidence changes.

> **Interview Phrasing:** "I evaluate whether the proposed service can be owned, changed, tested, and supported independently. I would look at actual co-change and release history rather than assume different teams need different microservices."
> *Plain English:* Independent releases matter only if teams can truly act independently.
> *Follow-up:* "What if the policy and purchase teams share one on-call team?" → Deployment separation may still work, but the operational autonomy benefit is smaller; quantify the support cost.

L11. Turn a proposed boundary into an explicit decision

Mapped: C037 — **combined capability / data / change / ownership judgment**. **Purpose/prerequisite:** Make a defensible architecture choice and identify when to reverse it. This applies a small piece of the F02 decision-record prerequisite. Requires L07–L10.

Terms: A **decision criterion** is a factor used to compare options. A **weight** is its declared importance. A **dependency** is something another part must provide. An **experiment** is a test designed to resolve uncertainty. A **revisit trigger** is a condition that causes the team to reconsider a decision. An ADR preserves context, decision, alternatives, and consequences [R09].

Worked example — boundary options:

```
Option A: one deployment            Option B: separate deployments
+--------------------------+        [Policy Service] --> [Policy data]
| Policy | Purchase        |                  ^
|   |        |             |                  | request
|  policy  purchase tables |        [Purchase Service] --> [Purchase data]
+--------------------------+
```

Fictional ADR example:

- **Context / business goal:** Deliver correct employee spend decisions while letting teams update rules safely.
- **Options: A** modular monolith with separate Policy and Purchase modules; **B** two independently deployed services.
- **Unknown facts:** Actual co-change rate, latency sensitivity, team support maturity, failure behavior, current data access paths.
- **Decision (conditional recommendation): A for the first release**, provided one release calendar meets the business need; enforce interfaces and separate table writers.
- **Consequences:** Lower runtime and incident complexity; coordinated deployment; extraction may later require work.
- **Evidence plan:** Module-boundary tests, deployment history, simulated policy dependency failure, design review with product/security/operations.
- **Revisit trigger:** Repeated policy-driven purchase release delays, verified independent scaling need, or unacceptable team coordination pain.
- **Owner:** Fictional platform architecture lead; product owners approve business trade-offs; service teams own implementation.

> **E11 — guided exercise:** Write your own ADR choosing **B**, including the strongest reason **A** might still be better. Include data owners, an outage test, a measurable trigger, and an accountable role. **Expected:** Reasoned alternatives and consequences, not merely "microservices scale better." **Hint:** Name the actual business friction you are buying relief from.

Failure/trade-off: A decision without a revisit trigger becomes dogma. A weighted matrix with invented precise scores can create false confidence. Use transparent ordinal ratings and a plan to gather evidence. Do not imply that cost, latency, or scale was measured when it was not.

> **Interview Phrasing:** "I would record the business driver, realistic alternatives, ownership, data boundaries, failure consequences, and what evidence would change the choice. Architecture decisions should be revisitable, not slogans."
> *Plain English:* Give a reason and a way to discover if that reason stops being true.
> *Follow-up:* "How do you know the boundary is wrong?" → Repeated coupled releases, cross-service data writes, hard-to-diagnose failures, or no real autonomy benefit.

L12. Tenants and isolation across service boundaries

Mapped: C038 — **tenant isolation**. **Purpose/prerequisite:** One business must not see or change another business's information. Requires L09 and the running-case tenant definition.

Terms: Software as a Service (SaaS) means software hosted and provided to customers as a service. **Multitenant** software serves more than one customer tenant. **Tenant isolation** prevents one tenant's actions or data access from crossing to another tenant without valid explicit authorization. **Authentication** checks the identity of a caller; **authorization** checks permission to act. An **identity context** is trusted caller information validated by the server. A **tenant identifier** is the key naming the business customer. A **noisy neighbor** is one customer whose resource use harms others. A **pool** shares infrastructure, a **silo** dedicates it, and a **bridge** mixes shared and dedicated resources. AWS describes these models and notes isolation must be enforced beyond login [R06, R07].

Worked example:

```
Acorn admin ---> [Shared API] -- enforce Acorn identity/permissions --> Acorn data
Birch admin ---> [Shared API] -- enforce Birch identity/permissions --> Birch data
                                  X
                        No Acorn --> Birch access
```

Step-by-step: 1. Server validates who is calling and which tenant memberships the identity has. 2. It selects the authorized tenant context. 3. Every data query and change is constrained to that context. 4. Internal services and reports **repeat/enforce** authorized scoping at each relevant boundary; passing a client-supplied `tenant_id` unchecked is unsafe. 5. Negative tests attempt Acorn access to Birch data. Tenant isolation is a security and correctness requirement no matter how many services are deployed [R07].

Isolation options and trade-offs:

Model	Advantage	Risk/cost	Example use case (hypothetical)
Pool: shared app / shared store with enforced tenant scoping	Resource efficiency	Incorrect query filters, noisy-neighbor and wide outage risk	Many similarly sized customers
Silo: dedicated store or stack for a tenant	More isolation choices, easier per-tenant performance separation	More deployments and management cost	High-isolation requirement verified with stakeholders
Bridge: share some layers; dedicate selected resources	Fine-grained trade-off	Increased routing and operational complexity	Shared identity + dedicated sensitive store

> **E12 — guided exercise:** Design four negative tests: Acorn user reading Birch policy, Acorn admin changing Birch limit, missing trusted tenant identity, Birch heavy reporting affecting Acorn. For each, state expected behavior and an owner. **Expected:** Unauthorized requests are denied; workload isolation is considered. **Hint:** A user interface hiding a button does not secure an API.

Failure/trade-off: Filtering only in the front end or trusting a URL tenant key causes cross-tenant leakage. Pooling is cheaper in some conditions but can increase blast radius; siloing is not a substitute for object-level authorization. **The exact security implementation belongs to T06.**

> **Interview Phrasing:** “I treat tenant isolation as a cross-cutting design constraint. I bind access to a validated tenant identity at each data boundary, test cross-tenant denials, and choose pooled or dedicated resources based on security, cost, and workload needs.”
> *Plain English:* Never let one company's information cross into another's area.
> *Follow-up:* “Does a separate database guarantee security?” → No; credentials, routing, APIs, backups, and administrator access can still cross boundaries.

L13. Model company hierarchy and policy inheritance

Mapped: C038 — **organizational hierarchy**. **Purpose/prerequisite:** A business customer can have many levels with different administrators and rules. Requires L12.

Terms: An **organization hierarchy** is a tree of units, such as company → division → department. A **parent** is above a child. **Inheritance** means a child receives a rule from a parent when no more specific valid rule exists. An **override** is a narrower rule that changes a broader one. An **entitlement** is permission for a user to act, such as editing limits for one department. A **scope** is the set of units that permission covers. A **cycle** is an invalid hierarchy where a unit eventually becomes its own ancestor.

Worked example (hypothetical rules, not real policy):

```
Tenant Acorn [base maximum 500 units]
  |
  +-- Headquarters [no override => 500]
  |
  +-- Sales        [override 200]
  |     +-- West Sales [inherits 200]
  |
  +-- Operations   [no override => 500]
```

Explicit assumption: Overrides can **only reduce** a parent maximum unless a separately authorized exception workflow exists. This is a design choice, not an industry payment rule. A Sales manager may edit only Sales and its children, **not** Operations. No hierarchy node may reference another tenant's node. A policy evaluator walks from the target unit toward the root, collecting applicable ceilings and choosing the strictest; it also checks who may change each rule.

Step-by-step: West Sales has no local limit; its effective ceiling is 200 (Sales) rather than 500 (Acorn root). An attempt to set Sales to 600 is rejected under this hypothetical rule. A user from Birch cannot attach a Birch department under Acorn.

> **E13 — guided exercise:** Under the stated hypothetical inheritance rule, find effective ceilings for Headquarters, West Sales, and Operations. Then answer: Who may change West Sales? What if Sales is moved under Operations? What if a unit has two parents? **Expected:** Results, permitted editor, revalidation after a move, and explicit open question about multi-parent semantics. **Hint:** Inheritance and permissions must be checked together.

Failure/trade-off: Copying the same effective limit to every child can make moves and updates inconsistent. Resolving the hierarchy on every request is more current but may cost lookup time. Consider storing an effective view with invalidation rules, but do not assert its freshness without tests. Cycles, stale caches, or mixed-tenant parent links must be detected.

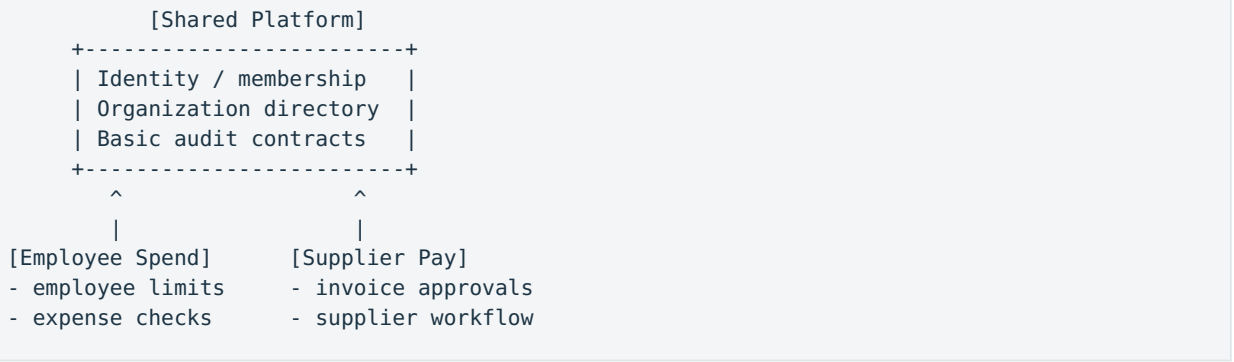
> **Interview Phrasing:** "I model the organization tree and policy inheritance explicitly. I would define override precedence, tenant boundaries, delegated administrator scopes, and how a hierarchy change updates effective rules."
> *Plain English:* Parent rules and local permissions must work together safely.
> *Follow-up:* "Should child departments be able to raise a parent limit?" → Only if the business defines an approved exception and the control owner accepts the risk.

L14. Shared platform capabilities versus product-specific behavior

Mapped: C038 — **shared platform capabilities** and **product-specific behavior**. **Purpose/prerequisite:** Reuse stable foundations without forcing unlike products into one giant rule engine. Requires L08 and L12–L13.

Terms: A **platform capability** is reused by several products, such as identity or organization membership. A **product capability** expresses a product's unique workflow. A **variation point** is a declared place behavior can differ. **Configuration** supplies values or approved choices without code changes. An **extension point** lets product-owned logic implement an agreed interface. A **shared kernel** is a small common model; if uncontrolled, it creates coupled releases. **Versioning** means supporting changes without unexpectedly breaking consumers.

Worked example:



Step-by-step: Both products use organization membership and verified caller identity. Employee Spend uses an employee limit policy. Supplier Pay has invoice approvals and supplier state that are **not** assumed to be identical to an employee purchase. A shared identity interface does not imply a shared payment lifecycle. Shared capabilities need an accountable provider, consumer contracts, upgrade policy, and support agreement. AWS SaaS guidance recommends considering different service load and isolation profiles during decomposition [R08].

> **E14 — guided exercise:** Classify each as **share**, **product-owned**, or **needs discovery**: tenant directory, user authentication, employee purchase eligibility, supplier invoice approval, reporting display, payment execution adapter. Provide reasons, a possible owner, and a risk for each. **Expected:** No forced sharing of unlike business rules. **Hint:** Different products may share infrastructure but not business semantics.

Failure/trade-off: Too much sharing becomes a central platform team bottleneck and may produce special-case flags everywhere. Too little sharing duplicates security controls and organization data. Prefer a stable shared core with explicit product extensions rather than hard-coded `if product == ...` branches scattered across the platform.

> **Interview Phrasing:** "I would centralize common, stable capabilities such as identity and organization membership. I would keep product-specific approvals and state where their business meaning differs, with well-defined integration contracts."
> *Plain English:* Share what truly means the same thing; keep different rules apart.
> *Follow-up:* "Would you centralize every rule in one rules engine?" → Only if the rules have a stable common model, governance, ownership, and a proven reuse benefit.

L15. Build, buy, or reuse: the boundary is also an ownership choice

Mapped: C038 — **shared platform decisions**, applying T01's "Prepare: build-versus-buy" note. **Purpose/prerequisite:** Decide whether a shared capability should be implemented by your team, provided by an internal platform, or purchased. Requires L14.

Terms: Build means the team develops and operates a capability. **Buy** means using an external product or service under a commercial agreement. **Reuse** means adopting an existing internal service. **Total cost of ownership (TCO)** includes acquisition, integration, licenses, operations, upgrades, support, and exit costs. **Vendor lock-in** means switching provider is costly. A **service level agreement (SLA)** documents an agreed service commitment. An **exit strategy** says how to change provider or bring a capability in-house. Cost guidance calls out build/buy, training, operating, and licensing costs together [R10].

Worked example — fictional identity choice:

Option	Potential benefit	Hidden risk to investigate	Owner to involve
Build identity capability	Tailored controls	High security maintenance and support burden	Security and platform owner
Reuse approved internal identity	Consistency, existing support	Shared release and capacity dependencies	Internal identity service owner
Buy managed identity service	Faster initial integration possible	Data handling, vendor availability, contract/exit risk	Procurement, security, architecture

Step-by-step: First define required identity functions and tenant integration. Second find approved existing capabilities. Third compare functionality, authorization boundaries, ongoing cost, resilience, portability, and support. Fourth name the owner of integration and vendor risk. **No option is assumed to be available or approved in a real company.**

> **E15 — guided exercise:** Write a five-line build/buy/reuse decision for a hypothetical organization-directory service. Include one risk about tenant separation, one contract dependency, one exit test, and an approval owner. **Expected:** A decision based on requirements and total lifecycle effects. **Hint:** A cheap license does not mean cheap integration.

Failure/trade-off: Buying a service does not transfer responsibility for correct access to customer data. Building a common platform for one consumer may waste effort. Reusing internal services without confirming support ownership creates an invisible dependency.

> **Interview Phrasing:** "For shared capabilities, I compare building, reusing, and buying against business fit, security, integration, ownership, lifetime cost, and exit risk. I prefer reuse when it truly satisfies requirements, not simply because a platform exists."

> *Plain English:* Shared capabilities still need an accountable owner and an evidence-based choice.
> *Follow-up:* "What if the vendor has no tenant-level export?" → That may be an unacceptable exit/data portability gap; clarify before committing.

---

16. Runnable Python demonstration — modules, tenants, hierarchy, and ownership

Why include code? T01 does not require production programming depth; a small, executable example helps make the boundary rules visible. It deliberately runs as **one process**. Its logical `Policy` and `Purchase` parts could later become separate services without changing the business meanings. Python syntax and tests follow official Python documentation [R11].

Prerequisites: A **dictionary** maps a key to a value; a **tuple** groups values (here, tenant and department); a **set** contains distinct members; a **function** is reusable code; `raise ValueError` rejects invalid input; `assert` checks expected output in a demo test. `None` means "no value." `while` repeats until the condition is false. `min(a,b)` chooses the smaller number. These are elementary Python constructs, not proprietary library features.

Run it: Copy the next complete block to `t01_boundary_demo.py` and run `python3 t01_boundary_demo.py`. It uses Python's standard features only. `python` may replace `python3` on systems where that is the installed Python 3 command. All data is intentionally in-memory, and all users, amounts, and company names are fictional.

```
PARENTS = {                                     # 01
    ("acorn", "root"): None,                     # 02
    ("acorn", "sales"): "root",                  # 03
    ("acorn", "west"): "sales",                  # 04
    ("birch", "root"): None,                     # 05
}                                               # 06
LIMITS = {                                      # 07
    ("acorn", "root"): 500,                     # 08
    ("acorn", "sales"): 200,                    # 09
    ("birch", "root"): 900,                     # 10
}                                               # 11
ACCESS = {                                      # 12
    "alice": {("acorn", "west")},                 # 13
    "bob": {("birch", "root")},                  # 14
}                                               # 15

def effective_limit(tenant, unit):              # 16
    current = unit                              # 17
    visited = set()                             # 18
    ceiling = None                              # 19
    while current is not None:                  # 20
        key = (tenant, current)                 # 21
        if key in visited or key not in PARENTS: # 22
            raise ValueError("Invalid hierarchy")# 23
        visited.add(key)                        # 24
        value = LIMITS.get(key)                 # 25
        if value is not None:                   # 26
            ceiling = value if ceiling is None else min(ceiling, value)  # 27
        current = PARENTS[key]                  # 28
    if ceiling is None:                         # 29
        raise ValueError("Missing base policy")  # 30
    return ceiling                              # 31
```

```
def evaluate(actor, tenant, unit, amount):      # 32
    if type(amount) is not int or amount <= 0:  # 33
        raise ValueError("Positive integer required")  # 34
    if (tenant, unit) not in ACCESS.get(actor, set()):  # 35
        return "DENY: no access"                # 36
    limit = effective_limit(tenant, unit)       # 37
    return "ALLOW" if amount <= limit else "DENY: limit"  # 38

assert effective_limit("acorn", "west") == 200  # 39
assert evaluate("alice", "acorn", "west", 200) == "ALLOW"  # 40
assert evaluate("alice", "acorn", "west", 201) == "DENY: limit"  # 41
assert evaluate("alice", "birch", "root", 1) == "DENY: no access"  # 42
assert evaluate("bob", "birch", "root", 500) == "ALLOW"  # 43
try:                                            # 44
    evaluate("alice", "acorn", "west", 0)      # 45
    raise AssertionError("Invalid amount accepted")  # 46
except ValueError:                              # 47
    pass                                        # 48
print("All demo checks passed")                 # 49
```

Line-by-line reading guide

Line	What that exact line does
01	Creates the `PARENTS` dictionary describing the hierarchy.
02	Creates Acorn's root, which has no parent.
03	Places Sales under Acorn's root.
04	Places West Sales under Sales.
05	Creates Birch's separate root.
06	Ends the hierarchy dictionary.
07	Starts a dictionary of configured ceilings.
08	Sets Acorn's top-level ceiling to 500 practice units.
09	Sets the stricter Sales ceiling to 200 units.
10	Sets Birch's unrelated base ceiling to 900 units.
11	Ends the ceilings dictionary.
12	Starts a dictionary of **prevalidated, fictional** actor permissions.
13	Allows Alice to request in Acorn's West unit only.
14	Allows Bob to request in Birch's root only.
15	Ends the actor-permissions dictionary.
16	Defines a function that finds the effective ceiling for one tenant/unit.
17	Starts from the requested department.
18	Creates a set to detect repeated units (cycles).

Line	What that exact line does
19	Starts without a known ceiling.
20	Moves up the hierarchy until there is no parent.
21	Makes a **tenant-scoped** lookup key.
22	Rejects a repeated or unknown unit instead of continuing unsafely.
23	Raises an explicit invalid-hierarchy error.
24	Remembers the current key for cycle checking.
25	Looks up the current level's optional ceiling.
26	Uses the ceiling only if this level defined one.
27	Sets the first ceiling or keeps the stricter minimum; this encodes our **hypothetical no-raise-override rule**.
28	Goes to the parent unit.
29	Checks whether any ceiling was found.
30	Rejects a hierarchy without an applicable base policy.
31	Returns the calculated effective ceiling.
32	Defines purchase-eligibility evaluation, separate from policy look-up.
33	Requires a positive integer amount; booleans and decimal floats are excluded in this toy model.
34	Explains a rejected input to the caller.
35	Checks **the actor's authorized tenant and unit** before accessing policy data.
36	Denies the cross-scope request.
37	Uses Policy's public function; it does **not** write the policy dictionary.
38	Returns allow only when the amount is at or below the effective limit.
39	Tests West Sales inheritance of the 200-unit ceiling.
40	Tests the exact allowed boundary.
41	Tests the amount immediately above the boundary.
42	Tests denial of one tenant's actor accessing the other tenant.
43	Tests a permitted Birch request.
44	Starts a test that expects an error.
45	Submits a zero amount.
46	Explicitly fails the test if no error occurred.

Line	What that exact line does
47	Catches the expected invalid-input error.
48	Intentionally does nothing because the expected error happened.
49	Prints a success message if all preceding tests pass.

What the example proves: Under its in-memory assumptions, rule inheritance, a basic permission check, and normal/boundary/invalid test cases behave as stated. The program can be explained from end to end.

What it explicitly does NOT prove: It is **not** a secure production service, financial authorization system, or complete multitenant design. It omits actual login/token verification, trusted identity transport, real role/entitlement delegation, persistence, audit logging, concurrency control, policy-version tracking, safe external financial actions, and enforcement by a database or gateway. `ACCESS` is a **fixture** assumed to have come from a trusted mechanism; do not accept it from a client request. `assert` is a demonstration test tool, **not** a production authorization control. These omissions are deliberate because their detailed treatment belongs to B03/T02/T06/T11.

> **E16 — guided exercise:** Change Acorn's Sales ceiling to 150, predict which assertions fail, update only the expectation for the effective limit and allowed boundary, and rerun. Then add a negative test denying Alice access to Acorn's root. **Expected:** You can explain why a rule change affects descendants while tenant/department permissions remain independent. **Hint:** Start at line 09; keep the Alice-versus-Birch denial.

17. Cumulative architecture exercise — the T01 practice task

> **Source requirement:** "Decompose the running case in two ways. Compare team ownership, data ownership, change frequency, latency, operational overhead, and migration difficulty." Complete this as one written architecture artifact **before** reading the key.

> **Hypothetical practice assumptions for this exercise only:** Two product teams are available; one platform support group operates common infrastructure. Both products initially release together. Finance can tolerate **up to 10 minutes** of reporting delay (**invented target**), but an employee needs an immediate eligibility response. A large tenant might demand dedicated storage, but no contract has yet confirmed this. There is no measured transaction volume, latency, error rate, team headcount beyond those stated, or cloud provider. **All of these are exercise assumptions, not employer facts.**

Part A — styles (C036): Draw two contrasting, realistic architectures: **Option A**, layered **modular monolith** with a deferred reporting event; and **Option B**, independent Policy/Purchase services with a reporting event. Show one API/contract or internal interface per meaningful boundary. Explain how SOA and event-driven design relate to (rather than replace) your two options.

Part B — boundaries (C037): Define candidate capabilities, team owners, consumers, write owners, and the data needed by each. For each boundary provide a one-sentence capability reason and one potential change-coupling risk.

Part C — compare and decide (C037): Compare the options against the **six binding dimensions**: team ownership, data ownership, change frequency, response latency, operational overhead, and migration difficulty. State which figures are unknown and which experiment would inform each unknown. Choose one option conditionally; write a short ADR with alternatives, consequences, and a reversal trigger.

Part D — tenants and hierarchy (C038): Draw Acorn and Birch isolation boundaries and Acorn's nested departments. Define where effective spend limits are decided, how ancestor rules apply, who can edit them, and how to test that Birch never sees Acorn data. Compare pool, silo, and bridge storage choices.

Part E — shared vs. product (C038): Classify identity, organization membership, employee eligibility, supplier invoice approval, transaction state, and reporting as shared or product-owned with a supporting reason. Select one shared capability for build/buy/reuse and identify accountable owners and vendor/data risks.

Submission checklist: Five labeled diagrams/tables are sufficient: (1) Option A, (2) Option B, (3) data + owner matrix, (4) tenant/hierarchy map, and (5) options/risks table; plus a compact ADR. Each should have a caption describing what it answers. *Expected outcome:* A coherent choice with explicit business assumptions, safe failure boundaries, and a plan to validate uncertain facts. A crisp 10-minute spoken explanation is good practice but **not** a formal X01 mock.

Hints (no full answer): Start with "which business fact is authoritative?"; distinguish "module" from "deployment"; explain what purchase processing does if policy rules cannot be checked; choose explicit rather than implied tenancy; do not equate a report with source transaction state.

18. Senior decision challenge — answer before reading §20

Source requirement (verbatim substance): “Two teams propose a single service versus separate spending-policy and transaction services. Pick a boundary based on ownership and change patterns, and say when you would reverse the choice.”

Interview prompt (fictional): Two teams must deliver the platform's spending policies and transaction processing. Team A proposes a **single deployable application** with two strongly separated modules. Team B proposes **separate Policy and Purchase services**. Both must protect Acorn/Birch data and let finance receive reports. Product leadership cares about fast rule changes; the operations team wants low support burden. During the interview, assume policy changes become more frequent and that a dedicated tenant store becomes a possibility, not a confirmed requirement.

Produce a senior answer covering all seven mandatory parts:

1. **Business goal:** What does the customer need, and how do you define success?
2. **Assumptions and missing facts:** What is known, fictional, unknown, or needs a business decision? Do not fabricate releases, costs, or performance results.
3. **Two or more realistic options:** Present a modular monolith and separated services; optionally include a staged hybrid.
4. **Decision and trade-off:** Choose now and say when it reverses. Explicitly cover all six comparison dimensions in §17.
5. **Failure, security, and data impact:** Where are authoritative policy/transaction records? What if Policy cannot answer? How is tenancy isolated and hierarchy enforced? What may be delayed in reporting?
6. **Evidence/experiment:** Which tests, change-history review, incident exercise, or measurement would validate your decision? Who defines acceptable thresholds?
7. **Owners and measurable outcomes:** Name business, technical, security, data, and operating roles; propose measurable **targets to agree**, not invented achievements.

Late constraint (after completing your initial answer): A product leader now requires **a new spending rule to be deployed without any Purchase release**. Revise your choice, explain whether it forces microservices or merely a better module/configuration boundary, and state the test that proves independent change. **Expected:** Defend the choice even if you keep the monolith; a new rule does not by itself prove separate deployment is required if separately managed data/configuration is sufficient.

No coaching in the real assessment: In a later requested interview session, answer without consulting §20; assess only after completion. That session is not started here.

19. Practice questions — do these without notes

Instructions: In 2–6 sentences per question, give (a) the definition, (b) a use in the running case, and (c) the downside or failure condition. Draw a small ASCII picture if a question asks for one. Afterward, check the separate answer key in §20. These are study questions, not claims about the actual hiring process.

ID	Checklist	Question
Q01	C036	What is the difference between a logical layer, a physical tier, a module, and an independently deployed service?
Q02	C036	Draw a simple N-tier purchase flow and explain one benefit and one limitation.
Q03	C036	How does SOA compare with microservices? Is a central enterprise bus required?
Q04	C036	When is a modular monolith a better fit than microservices? What would make you change?
Q05	C036	Why is event-driven architecture not automatically a replacement for a synchronous purchase decision?
Q06	C037	Propose capability boundaries for Policy, Purchase, and Reporting. What business meanings separate them?
Q07	C037	Who should be able to update policy limits, purchase state, and report projections, and why?
Q08	C037	What evidence would justify separate Policy and Purchase deployments? What evidence argues against?
Q09	C037	Write four necessary sections of an ADR for this boundary, plus a measurable revisit trigger.
Q10	C038	Explain pooled, siloed, and bridge tenant isolation, including a cross-tenant test and noisy-neighbor consideration.
Q11	C038	Acorn root has ceiling 500, Sales has 200, West Sales has no local ceiling. Under our invented rule, what applies? Who may edit it?
Q12	C038	Which capabilities should Employee Spend and Supplier Pay share? Which differences must remain product-owned?
Q13	C036	Can a layered modular monolith publish events? Can microservices use layers internally? Why?

ID	Checklist	Question
Q14	C037	Purchase and Policy always release together, even as separate services. What is probably wrong or missing?
Q15	C038	A shared organization service may be bought from a vendor. Which data, security, support, and exit questions must you answer?

Interview follow-up drills:

- **IF1 — C036:** "Your diagrams show five services. Why five rather than two?" Answer using capability cohesion and actual need for independent deployment.
- **IF2 — C037:** "Why is spending policy separate from transaction processing?" Answer with rule ownership, authoritative records, co-change evidence, and a reversal condition.
- **IF3 — C038:** "An Acorn employee can guess a Birch transaction ID. What stops access?" Answer with server-side validated tenant/unit authorization and cross-boundary tests, not interface hiding.

20. Separate answer key — exercise solutions and model decisions

> **Use only after an attempt.** The solutions below are **illustrative**, not the only valid architectures. Different assumptions can produce a different defensible choice. A learner's own experience must never be invented to match these answers.

P0 prerequisite-check solution

Expected: Context includes external people/systems and the platform; component separates Policy and Purchase responsibilities; sequence places the policy evaluation before the customer-facing result. State that the Purchase component owns request processing, while Policy owns rule meaning. No diagram alone proves correctness.

Guided exercises E01–E07 — architecture styles (C036)

- **E01:** Single process: `[Policy module | Purchase module]`, one shared deploy boundary, logical module boundary, explicit owned tables. Two processes: `[Policy process] <--contract/network--> [Purchase process]`, two deployment and failure boundaries. In both designs, trust and data ownership must be stated separately.
- **E02:** Parse and render in Presentation; compare limit in Business; fetch policy in Data Access. (a) one backend plus a database tier; (b) separate web, business, and database tiers. **A logical layer does not require its own tier.**
- **E03:** Example contract: `ResolveUserMembership(trusted_user_context) -> permitted_org_units`. Consumers: Employee Spend and Supplier Pay. Provider: shared identity/organization platform team. The contract must describe success, denial, and how changes are communicated. Neither product may write the provider's tables.
- **E04:** Four enforceable rules: Policy exposes documented functions; Purchase writes only Purchase-owned records; dependency checks prohibit direct imports of Policy's private internals; separate tests reject cross-module table updates. One deployable application simplifies local tests/operations; coordinated releases are its downside.
- **E05:** Possible justifications: measured policy-only release demand, truly independent scale, distinct accountable on-call ownership. Added responsibilities: service contract compatibility, failure/timeouts and health monitoring, secure interservice access, independent deployment tests, controlled data propagation. Without evidence, the split may be premature.
- **E06:** `EvaluatePurchase` and `SendNotice` are **commands**; `PurchaseRecorded` and `LimitChanged` are **events** if those facts have already occurred. The employee-facing eligibility decision usually needs a direct result or explicitly managed pending outcome. Notification/report updates may be asynchronous if the freshness target permits. No command/event label alone proves guaranteed processing.
- **E07:** Illustrative **judgments, not measured facts**: A = easier operation, low communication overhead, coordinated release; B = potentially stronger deploy/team autonomy, higher network and support overhead. Both can assign exclusive write owners. Choose A until co-change and support evidence justify B; revisit after verified team or latency pain. The important output is the **reason and reversal test**, not invented numeric ratings.

Guided exercises E08–E11 — service boundaries (C037)

- **E08:** Verbs: set rule, evaluate eligibility, record request, retrieve status, reconcile report. Candidate capabilities: Policy, Purchase, Organization, Reporting, External Execution. Policy defines eligibility; Purchase manages request lifecycle. Neither must be its own deployment yet.

- **E09:** Only Policy writes rule state; only Purchase writes authoritative purchase state; only Directory changes membership; Reporting maintains a derived projection. A Reporting writer changing `purchase.status` bypasses source ownership. Detect with privilege tests, interface review, and a negative integration test.
- **E10:** Under the **invented** high-change Policy case, separate deployment might reduce coordinated releases and let controls ownership act independently. It may still be wrong if changes always require Purchase logic updates or if the operations team cannot support two services. Owners: policy/purchase product owners for business needs, engineers for co-change evidence, operations for support readiness.
- **E11:** Example choice B ADR: **Context:** rule updates blocked by combined releases (hypothetical; verify). **Decision:** separate Policy and Purchase runtimes; Policy exclusively owns rules; Purchase owns requests. **Alternative A:** modular monolith avoids network and operations risk. **Consequences:** independent change, contract and outage burden. **Test:** deploy a backward-compatible rule update with no Purchase release, inject Policy unavailability, validate safe deny/pending behavior and isolated tenant queries. **Revisit:** repeated cross-service changes or excess incident load; designate owners and target for review. A reasonable answer may still choose A with a credible validation plan.

Guided exercises E12–E16 — tenant, hierarchy, sharing, and Python (C038)

- **E12:** (1) Acorn reads Birch rule → deny, no data returned; (2) Acorn admin writes Birch limit → deny, no persisted change; (3) trusted tenant context missing → fail closed; (4) Birch heavy reports → inspect per-tenant resource use, quotas, scheduling or dedicated capacity if warranted. Security test owner, data owner, and operations owner should each be named. Distinguish **confidentiality isolation** from **capacity isolation**.
- **E13:** Effective ceilings: Headquarters 500, West Sales 200, Operations 500. Under this **fictional delegation rule**, a Sales-scoped administrator authorized for West Sales (or an explicitly authorized ancestor) can edit within its scope; ordinary employees cannot. Moving Sales under another parent requires validating effective policy and authorization. Multi-parent hierarchy needs **business clarification**; the toy tree forbids it, and an arbitrary first-parent choice is unsafe. A Sales ceiling of 600 would violate the current hypothetical no-raise rule.
- **E14:** Likely shared: tenant directory, authenticated identity, organization membership. Product-owned: employee eligibility; supplier invoice approval. Needs discovery: reporting presentation and execution adapter, because common underlying mechanisms may have different rail/product semantics and service owners. A valid answer names a consumer and explains ownership for each item.
- **E15:** Example "reuse": Adopt a verified internal organization-directory service *only if* it represents the required tenant and unit hierarchy. The team owns integration and negative authorization tests; the directory team owns its authoritative data. Check contract lifecycle, outage mode, data export/deletion, security review, support response, and migration/exit path. If these conditions fail, compare build and buy again.
- **E16:** After changing Sales to 150, `effective_limit("acorn", "west")` becomes 150. The former `200 == "ALLOW"` assertion fails and needs a test of `150 == "ALLOW"`; `201 == "DENY: limit"` still passes. Add `assert evaluate("alice", "acorn", "root", 1) == "DENY: no access"`. The Birch denial must remain. Do **not** change access permissions simply to make tests green.

Self-test Q01–Q15 — compact model answers

ID	Essential correct answer and expected trade-off
Q01	Layer = logical job grouping; tier = separately deployed/runtime part; module = unit of related code; service = owned capability behind a contract, optionally independently deployed. Distinguish code, data, runtime, and trust.
Q02	Browser → presentation → business rule → data access → database. Simple separation; capability changes can still touch many layers or tiers.

ID	Essential correct answer and expected trade-off
Q03	SOA promotes contract-based service reuse across applications; microservices emphasize bounded capability autonomy and independent operations. SOA does **not** mandate a bus. Shared governance can slow change.
Q04	Favor modular monolith where one release/team model is acceptable and operations simplicity matters. Revisit when independent releases, scales, or owners have measured value.
Q05	An event says a fact happened. It can update reports later, but cannot by itself promise an immediate decision; delayed/duplicate processing and freshness are explicit risks.
Q06	Policy owns rule meaning; Purchase owns the request and its status; Reporting owns derived views. Boundaries are justified by different capabilities, not controller names.
Q07	Policy updates policies; Purchase updates request facts; Reporting updates derived reports only. Shared writable state creates hidden coupling and ambiguous authority.
Q08	Support: measured independent change/scale, compatible contract, distinct owners. Oppose: high co-change, operational immaturity, unacceptable network/dependency failure or cost.
Q09	Context/goal; options; decision; consequences; plus a review trigger such as repeated blocked rule releases validated against change records. Assign an ADR owner.
Q10	Pool shares resources; silo dedicates them; bridge mixes. All require access enforcement. Test Acorn denial of Birch data; pooled designs need noisy-neighbor measures.
Q11	West Sales inherits effective 200 under the explicit strictest-ceiling assumption. Only an authorized admin with the correct tenant/unit scope may edit; validate child/ancestor permissions.
Q12	Share identity/directory where semantics are common; retain employee eligibility and supplier invoice approval under their domain owners. Sharing a platform does not erase product differences.
Q13	Yes and yes. Layers and modules describe internal organization; event-driven describes interaction; microservices describe autonomous service deployment. Styles can combine.
Q14	Separate deployment may not give autonomy; contracts or shared writes force coordinated changes. Look at co-change records, boundary responsibilities, and whether a simpler single deployment is preferable.
Q15	Confirm tenant scoping, trusted identity, data authority, export/deletion and residency needs, contract evolution, failure SLA, support/on-call, price over time, and a tested exit path.

Follow-up drill answers: IF1: number of services should follow candidate business contexts and independence evidence, not class count. IF2: policy owns rules and changes, purchase owns request state, and separate deployment is conditional on measured autonomy; reverse if co-change dominates. IF3: use server-side validated identity, per-tenant and per-object authorization, and negative tests all the way to storage/reporting; never trust guessed identifiers or client-side filtering.

Cumulative exercise — one model solution, NOT a mandatory answer

Two candidate decompositions:

```
OPTION A: MODULAR MONOLITH + REPORTING EVENT

[Employee/Admin API]
          |
+---------v-----------------------------------------------+
| One deployable application                               |
| [Org module] --membership--> [Policy module]             |
|                                     ^                    |
| [Purchase module] ----evaluate--------+                  |
|       |                      owned policy tables         |
|   owned purchase tables                                 |
+-------+-----------------------------------------------+
        | published fact (after authoritative record)
        v
   [Event channel] ---> [Reporting consumer + projection]

OPTION B: INDEPENDENT POLICY + PURCHASE SERVICES

[Employee/Admin API] ---> [Purchase Service] ---> [Purchase data]
                                |
                                | policy decision contract
                                v
                          [Policy Service] ---> [Policy data]
                                ^
                                | member lookup
                          [Org/Identity capability]
[Purchase Service] --> [Event channel] --> [Reporting]
```

Common rules for both: Access to authoritative records is tenant-scoped; policy owns rules; purchase owns request state; reporting owns only derived state. The external payment capability is an integration dependency, **not assumed to share an internal database or financial state machine**. If Policy data cannot be evaluated, **hypothetical safe option** is deny or explicitly mark pending according to business-approved controls; do not silently allow financial actions. The precise fail-closed/fallback behavior must be chosen with risk/product owners.

Six-dimension comparison:

Dimension	Option A	Option B	Unknown fact to verify
Team ownership	Logical owners, shared release	Separate teams may release alone	Actual team/on-call responsibility
Data ownership	Owned tables in one store possible	Distinct owner stores	Current cross-writes, migration paths
Change frequency	Coordinated deployment, possibly configurable rule updates	Can deploy independent policy logic	Real co-change and release history
Latency	Local policy call usually avoids network hop	Extra network call; can fail	User budget and tested dependency latency
Operational overhead	Lower initial moving-parts count	More deployments, logs, alerts, incident roles	Operations readiness and measured cost

Dimension	Option A	Option B	Unknown fact to verify
Migration difficulty	No runtime split now; future extraction effort	Extraction needed now; phased rollout	Legacy data coupling and consumer contracts

Illustrative decision: Start with A *if* one release process meets the business requirement, while retaining explicit data/module boundaries and an event-based reporting interface. Review after observing releases and business delay. If business truly requires independently deployable policy logic, adopt B after contract, failure, authorization, and tenant-isolation tests. A strong candidate may choose B immediately when sufficient evidence and team maturity are provided. Neither is always superior.

Tenant/hierarchy plan: Acorn and Birch get separate logical tenant namespaces and verified access rules. Acorn's West Sales inherits the strictest ceiling among its ancestors, with department-scoped admin permissions. Test attempted cross-tenant reads/writes, invalid parent links, hierarchy moves, and high reporting load. Consider silo/bridge **only after** isolation and cost requirements are confirmed; do not equate a dedicated database with full isolation.

Shared/product mapping: Shared identity and directory; employee policy and supplier approval stay product-specific; reporting can reuse infrastructure but keep product-defined views and source authority. Named owners: fictionally, identity/directory platform lead, controls product owner, purchasing product owner, finance reporting owner, security approver, and operations on-call lead.

Short ADR for the model: "Adopt two owned modules in one deployable application as initial architecture; expose private interfaces, keep write owners distinct, publish report facts reliably using a to-be-designed T02/T04 mechanism. Accept coordinated deployments to reduce current operational risk. Revisit when measurable policy release delays or tenant isolation requirements justify a separate service or store. Require a backward-compatible contract test, cross-tenant negative tests, and a reviewed failure path before extraction." This is a **fictional recommendation**, not an industry mandate or a company decision.

Senior decision challenge — example reasoning, not a script to memorize

1. **Goal:** Correct tenant-scoped spend decisions and maintainable rule changes; confirm target response behavior with Product and Risk.
2. **Unknowns:** Frequency of rule deployments, real co-change, team autonomy, response-time budget, vendor/storage constraints, support ability; none is measured in the prompt.
3. **Options:** Modular monolith with strict modules; separate services; or modular monolith first with a tested extraction seam.
4. **Choice:** Prefer modular monolith initially **only while** releases are acceptable; trade off independent deployment for simpler failure handling. If a verified business need requires no Purchase release for Policy code, prefer separate services or an externalized/configurable policy mechanism meeting the same control objective.
5. **Risks:** Policy data owner stays exclusive, Purchase alone owns request state; missing Policy decision means no unapproved purchase action; tenant checks on every access path; reports may lag only within a business-approved target. Do not claim external financial success from a policy decision.
6. **Evidence:** Review 3–6 representative changes **as a proposed sample, not actual history**, test no-Purchase-release configuration, load and failure injection at the policy boundary, and try Acorn→Birch negative access. Agree pass criteria before running tests.
7. **Owners/outcomes:** Product owner approves allowed business behavior; controls owner validates rules; engineering leads own services; security owner approves isolation; operations owns failure response; data owners own authoritative facts. Proposed metrics: rule change lead time, rate of joint releases, failed cross-tenant authorization tests (target zero allowed), purchase decision errors, and report freshness. **Targets require agreement; no achieved values are asserted.**

Late-constraint response: First ask whether “new rule without Purchase release” means changing **configured values** or deploying **new executable rule logic**. For configuration, a modular monolith can sometimes meet the need through an independently managed, approved rule-data change. If independent policy executable releases are explicitly required, a separate Policy service becomes more compelling; it still needs contract compatibility, tenant checks, and a safe unavailable-policy behavior. Reversing is appropriate if policy/purchase changes repeatedly co-occur or the split creates disproportionate incident burden. **The most important senior behavior is distinguishing business necessity from a technology label.**

21. Assessment rubric and completion criteria

Standard v3 scoring rubric — do not soften the threshold

Score	Observable capability
1/5	Cannot accurately explain the concept even with prompts.
2/5	Recognizes terms but needs help applying them; assumptions or risks are missing.
3/5	Proposes a workable design but weakly compares alternatives or failure modes.
4/5	Independently explains, applies, compares realistic options, and defends trade-offs with checks or evidence.
5/5	Meets 4/5 and adapts to changed constraints; identifies second-order effects for business, data, security, operations, and delivery where relevant.

Assess these dimensions separately:

Dimension	Evidence of a 4/5 answer in T01
Business framing	States customer journey, business outcome, constraints, and success measures rather than starting with an architecture label.
Technical correctness	Correctly distinguishes layers, tiers, SOA, modular monoliths, microservices, events, authoritative data, tenant scope, and inheritance.
Trade-offs	Compares at least two implementable designs using all six required comparison dimensions and names a reversal trigger.
Failure/security/data risk	Names unavailable-policy behavior, write owners, cross-tenant controls, hierarchy effects, and reporting-lag implications.
Evidence	States missing measurements and at least one experiment or review with a clear pass/fail result for each material uncertainty.
Execution/ownership	Assigns business, service, security, data, and operational responsibilities; describes how a decision and contract become tested work.
Communication	Gives a structured, clear, natural answer that defines terms, separates assumptions from facts, and responds to a changed requirement.

Senior decision test pass: At least 4/5 on every applicable dimension. Do **not** average a low security or technical score into a passing total. T01 is not the X01/X02 mock; their separate two-mock requirement is unchanged.

Item-by-item pass gate

For **each** of **C036, C037, and C038**, require **all three forms of independent evidence**:

- **Explain without notes:** define the terms and purpose accurately.
- **Apply without notes:** build the small lesson exercises and corresponding cumulative artifacts; code is optional evidence for T01 but useful.
- **Defend without notes:** name a realistic failure, compare alternatives, justify the selected trade-off, and show what would change the choice.

Card T01 completion requires all:

1. P0 prerequisites demonstrated or marked as an open prerequisite needing F02/foundations review.
2. E02–E15 **and** the C036–C038 portions of the cumulative task attempted and reviewed against the key; E01 and E16 strengthen but do not replace a missing Core item. Every failed exercise has a corrected outcome.
3. **C036, C037, and C038 individually pass** explain/apply/defend without notes.
4. The full §18 senior test scores **≥4/5 in each applicable dimension**.
5. Any unresolved factual assumption or operational risk is recorded with an owner and a verification action.
6. The learner can speak a coherent answer to "Why is your proposed policy service separate from transaction processing?" and also defend keeping it together under different conditions.

Blank learner scorecard — do not pre-fill success

Evidence	Status now	Evidence / score / follow-up
Prerequisite P0 check	**Not assessed**	—
C036 explain / apply / defend	**Not assessed**	—
C037 explain / apply / defend	**Not assessed**	—
C038 explain / apply / defend	**Not assessed**	—
Cumulative design and correction	**Not assessed**	—
Senior business framing	**Not assessed**	—
Senior technical correctness	**Not assessed**	—
Senior trade-offs	**Not assessed**	—
Senior failure/security/data	**Not assessed**	—
Senior evidence	**Not assessed**	—
Senior execution/ownership	**Not assessed**	—
Senior communication	**Not assessed**	—
Reassessment after weaknesses	**Not assessed**	Record date, change, and new evidence

Completion decision: Not yet passed. This file contains teaching content and reference answers, **not actual learner performance**. When a review session is requested, score observable answers, log weak dimensions, correct them, and retest without notes.

22. Official primary references and verification record

Verification method: The URLs below were opened or returned as official first-party documentation or as a published issuing-body standard during web verification on **2026-10-24**. They support the **general technology statements** cited in lessons. They do **not** validate the fictional running case, any company architecture, expected job duties, exact interview process, or learner performance. Vendor architecture guides provide documented practices and trade-offs, **not binding universal standards**. The OASIS SOA reference model is an issuing-body standard.

Key	Official primary source	What it supports	Checked
[R01]	Microsoft Azure Architecture Center — Architecture styles	Layer/N-tier organization; difference among styles; choosing against constraints	2026-10-24
[R02]	OASIS — Reference Model for Service Oriented Architecture v1.0	Abstract SOA concepts without requiring a specific protocol/bus	2026-10-24
[R03]	Microsoft Azure Architecture Center — Microservices architecture style	Bounded contexts, business capabilities, autonomous services, private data, trade-offs	2026-10-24
[R04]	Microsoft Azure Architecture Center — Use domain analysis to model microservices	Deriving boundaries, avoiding arbitrary team-chart splits, revisiting boundaries	2026-10-24
[R05]	Microsoft Azure Architecture Center — Event-driven architecture style	Producers, consumers, broker, asynchronous processing, event design limitations	2026-10-24
[R06]	AWS — SaaS Tenant Isolation Strategies: pool, silo, bridge	Shared, dedicated, and hybrid tenant deployment trade-offs	2026-10-24
[R07]	AWS SaaS Lens — Isolation mindset	Tenant isolation beyond authentication and authorization at the entry point	2026-10-24
[R08]	AWS SaaS Lens — General design principles	Tenant-aware decomposition by load and isolation profile	2026-10-24
[R09]	AWS Prescriptive Guidance — Architectural decision record process	Context, decision, alternatives, ownership, consequences, and decision lifecycle	2026-10-24
[R10]	Microsoft Azure Well-Architected Framework — Cost optimization principles	Build-versus-buy, lifetime costs, operations, training, vendor selection trade-offs	2026-10-24
[R11]	Python official tutorial and Python official modules tutorial	Executable Python functions, dictionaries, modules, and basic syntax	2026-10-24
[R12]	Google Cloud Well-Architected Framework — Promote modular design	Modular design regardless of monolith/microservices packaging	2026-10-24 (publisher page indicated last reviewed 2026-10-22)
[R13]	Microsoft Azure Architecture Center — Organize Azure resources in multitenant solutions	Tenant isolation choices need not be identical at every layer	2026-10-24

Important interpretation: The inheritance rule “a child may only lower a parent's ceiling,” the two-tenant case, synthetic spending limits, grouping of products into specific services, 10-minute reporting target, and example policies are explicitly **Hypothetical practice assumptions**. They are *not* statements found in [R01]–[R13]. Build/reuse/buy and modular-monolith-first are **conditional recommendations**, not source mandates.

Source requirement provenance: The supplied v3 topic map, especially its “Learning rules,” “Communication and Teaching Guidelines,” “Binding learning-guide production contract,” “Verification and evidence protocol,” and “T01 — Service boundaries and architecture styles” subsections, governs this document. The source file has no verified American Express job requisition; this guide does not attempt to invent one.

23. Final completeness and verification audit

A. Syllabus-to-guide audit

Required T01 coverage	Lessons	Exercise/test	Document status	Learner status
C036 — Layered/ multi-tier	L02, L07	E02, E07, Q01–Q02, Q13	**Included**	Not assessed
C036 — SOA	L03, L07	E03, Q03	**Included**	Not assessed
C036 — Modular monolith	L04, L07	E04, Q04, Q13	**Included**	Not assessed
C036 — Microservices	L05, L07	E05, Q03–Q04, Q13	**Included**	Not assessed
C036 — Event-driven designs	L06, L07	E06, Q05, Q13	**Included**	Not assessed
C037 — Capability derivation	L08, L11	E08, E11, Q06, Q09	**Included**	Not assessed
C037 — Data boundaries	L09, L11	E09, E11, Q07	**Included**	Not assessed
C037 — Ownership and change patterns	L10–L11	E10–E11, Q08, Q14	**Included**	Not assessed
C038 — Tenant isolation	L12, §16	E12, E16, Q10	**Included**	Not assessed
C038 — Organizational hierarchy	L13, §16	E13, E16, Q11	**Included**	Not assessed
C038 — Shared platform capabilities	L14–L15	E14–E15, Q12, Q15	**Included**	Not assessed
C038 — Product-specific behavior	L14	E14, Q12	**Included**	Not assessed
T01 practice: two decompositions, six criteria	L07, L11, §17	E07, E11, §17 parts A–C	**Included**	Not assessed
T01 senior decision test / reversal choice	§18	§18 all seven parts + late constraint	**Included**	Not assessed

Required T01 coverage	Lessons	Exercise/test	Document status	Learner status
Per-item six-step instruction	L02–L15	Purpose, terms, worked steps, exercise, failure/trade-off, interview + follow-up in each	**Included**	Not assessed
Separate answer key, rubric, scorecard	§§20–21	E01–E16, Q01–Q15, cumulative and senior model	**Included**	Not assessed

B. Missing, not verified, or intentionally excluded

Issue	Status	Consequence / next action
Missing T01 Core checklist content	**None identified**	All C036, C037, C038 components have distinct lesson and assessment evidence in the guide.
Syllabus label mismatch "F02 (C036–C038)"	**Resolved by authoritative mapping**	F02 owns C019–C022; the requested item IDs belong to T01. Full F02 training remains in its own card.
Actual learner understanding and pass result	**Not verified**	Guide cannot claim the learner completed or passed exercises; review and retest after a requested teaching/assessment session.
Exact American Express role posting, interview rounds, internal stack, private service boundaries	**Not verified / not supplied**	Do not present fictional designs as company facts. If a verified posting is later provided, compare it to this syllabus without silently changing core scope.
Actual tenant contract, hierarchy semantics, rule exception process, accountabilities, SLAs	**Not verified; deliberately hypothetical**	Confirm with business, risk, security, legal, and operations before real implementation.
Measured latency, load, release frequency, co-change rate, failure rate, cost, support capacity	**Not verified; no data supplied**	Gather real evidence before selecting services or declaring benefits. Practice numbers are labeled assumptions.
Production security and data correctness of Python demonstration	**Not verified / explicitly not claimed**	Demo is in-memory and lacks identity validation, database permissions, audit, concurrency, durable changes, and financial execution; see B03, T02, T06, T11.
Detailed event delivery, exact-once semantics, retries, saga, broker recovery	**Intentionally excluded per T01 scope**	Study T02/T04 when reached; only C036 conceptual comparison is required here.
Vendor-specific cloud selection, procurement quotes, regulatory compliance claims	**Not verified; not needed for T01**	Obtain official organizational and regulatory evidence if a real project requires them.
F02 formal views/reviews, full API/security/payment curricula	**Prerequisite or other cards, not missing T01 lessons**	Use F02, T03, T06, D02 as separate syllabus units.
Official citations	**Verified for general conceptual claims as of 2026-10-24**	Links may change later; recheck primary publishers before using this guide for a real decision or interview with new claims.

Issue	Status	Consequence / next action
Fictional case and model ADR	**Explicitly hypothetical, not externally verifiable**	Never portray as prior work history or internal company architecture.

C. Final acceptance checklist (document-level)

- ☑ Authoritative syllabus read; T01 scope preserved; F02/C036 naming conflict flagged.
- ☑ Separate explicit coverage for **all five C036 styles** and **all four C037 decision factors**.
- ☑ Separate explicit coverage for **tenant isolation, hierarchy, shared platform, and product-specific behavior** under C038.
- ☑ Prerequisites before dependencies; basic vocabulary and expansions; progressive short lessons.
- ☑ ASCII diagrams; runnable Python example with a **line-by-line** explanation, scenario and test limitations.
- ☑ Guided exercises, expected results, hints, failure modes, alternatives, and natural Interview Phrasing with explanations/follow-ups.
- ☑ Cumulative two-decomposition comparison covering the six required dimensions.
- ☑ Separate practice questions and **separate answer key**, including senior model decision and late constraint.
- ☑ Senior seven-part decision test and original map's 1–5 scoring standard and **≥4 per applicable dimension** pass gate.
- ☑ Primary official references, verification date, clear fact/assumption/recommendation/source labels.
- ☑ Gap register, verification limitations, learner scorecard marked **not assessed**.
- ☐ **Learner has demonstrated mastery:** Not evaluated; requires a separate requested session.
- ☐ **Company-specific facts and target quality measures validated:** Not available; no values fabricated.

Document verdict: T01 syllabus coverage is complete at document level. Learner readiness, production suitability, and company-specific accuracy are not proven. This is a reference guide, not a certification, official interview guide, or verified implementation plan.